VGG16_FeatureExtractor_SVM_Kas_Kod_Aciklama_Raporu.pdf - local repository text copy

VGG16_FeatureExtractor_SVM_Kas_Kod_Aciklama_Raporu.pdf - local repository text copy
Sayfa 1
VGG16 FEATURE EXTRACTOR + SVM
KAŞ BÖLGESİ DEEPFAKE DENEYİ
Kod Açıklama ve Teknik İşleyiş Raporu
Kaynak notebook:
VGG16_FeatureExtractor_SVM_Kas_Deney1_FIXED (1).ipynb
Tarih: 8 Ağustos 2026
Bu rapor, notebook içindeki kodun ne yaptığını hücre ve bileşen bazında açıklar. Sonuçların başarım yorumundan ziyade, veri akışı, modelleme mantığı, kayıt mekanizması ve teknik kararların açıklanmasına odaklanır.
Sayfa 2
İçindekiler
1. Raporun Amacı ve Notebookun Genel Görevi
2. Uçtan Uca Veri Akışı
3. Hücre 0 - Colab ve Bağımlılık Kurulumu
4. Hücre 1 - local source, Dizinler, Konfigürasyon ve Logging
5. Veri Envanteri ve Video Kimliği Çıkarımı
6. Veri Sızıntısı / Split Kontrolleri
7. VGG16 Feature Extractor Yapısı
8. Görüntü Ön İşleme, Batch İşleme ve Feature Cache
9. SVM Hiperparametre Seçimi
10. Final Modelin Train + Validation ile Eğitilmesi
11. Test Tahminleri ve Frame-Level Metrikler
12. Video-Level Değerlendirme Mantığı
13. Grafiklerin Üretilmesi
14. Deney Metadata ve Tekrarlanabilirlik Dosyaları
15. Output Manifest ve Dosya Bütünlüğü
16. Üretilen Dosyaların Anlamı
17. Kodun Güçlü Tasarım Noktaları
18. Koddan Doğrudan Görülen Sınırlamalar ve Dikkat Noktaları
19. Özet Pseudocode
Sayfa 3
1. Raporun Amacı ve Notebookun Genel Görevi
Notebook, kaş bölgesine kırpılmış REAL ve FAKE frameler üzerinde iki aşamalı bir sınıflandırma sistemi kurar. İlk aşamada ImageNet ağırlıklarıyla yüklenen VGG16 ağı yalnızca derin özellik çıkarıcı olarak kullanılır. İkinci aşamada bu 512 boyutlu özellik vektörleri StandardScaler ile ölçeklenir ve SVM sınıflandırıcısına verilir. Kod, mevcut train/val/test ayrımını yeniden üretmez veya karıştırmaz; klasörlerde
bulunan ayrımı doğrudan kullanır.
Eyebrow frame -> VGG16 (ImageNet, include_top=False, pooling=avg)
-> 512-D feature -> StandardScaler -> SVM -> REAL / FAKE
Aynı notebook yalnızca model eğitmekle kalmaz; veri envanteri çıkarır, olası split sızıntısını kontrol eder,
feature cache üretir, hiperparametre araması yapar, final modeli kaydeder, test tahminlerini ve metrikleri
dışa aktarır, grafikler üretir ve deney ortamını yeniden oluşturmayı kolaylaştıran metadata dosyaları hazırlar.
2. Uçtan Uca Veri Akışı
Şekil 1. Notebookun uçtan uca teknik akışı.
☐ Girdi: Kaş klasörü altındaki REAL/FAKE ve train/val/test frameleri.
☐ Özellik çıkarımı: Dondurulmuş VGG16, sınıflandırma başlığı olmadan kullanılır.
☐ Model seçimi: SVM adayları train üzerinde eğitilir, validation üzerinde karşılaştırılır.
☐ Final eğitim: En iyi parametrelerle train + validation birleştirilir.
☐ Final değerlendirme: Test seti yalnızca son aşamada kullanılır.
☐ Çıktılar: Model, scaler, metrikler, tahminler, grafikler, feature cache, log ve deney metadata dosyaları.
3. Hücre 0 - Colab ve Bağımlılık Kurulumu
İlk kod hücresi çalışma ortamını hazırlar. Scikit-learn, PyYAML, joblib ve tqdm paketleri kurulur; ardından
TensorFlow/Keras, NumPy, pandas, PIL ve matplotlib gibi kütüphaneler içe aktarılır.
```
!pip -q install "scikit-learn>=1.4,<2" pyyaml joblib tqdm
```
Bileşen	Kod içindeki görevi
TensorFlow / Keras	VGG16 modelini yüklemek ve görüntülerden derin özellik çıkarmak.
scikit-learn	StandardScaler, SVC ve değerlendirme metrikleri.
PIL	Görüntü açma, RGB dönüşümü ve yeniden boyutlandırma.

NumPy Feature matrisleri, etiket dizileri ve sayısal işlemler.

pandas Dataset envanteri, arama sonuçları ve CSV çıktıları.
Sayfa 4
joblib Final SVM ve scaler nesnelerini diske kaydetmek.
matplotlib Confusion matrix, ROC ve precision-recall görsellerini üretmek.
hashlib Output manifest için SHA-256 dosya özeti üretmek.
logging Deney akışını training.log dosyasına yazmak.
Tekrarlanabilirliği artırmak amacıyla Python, NumPy ve TensorFlow için 42 seed değeri ayarlanır. Ayrıca TensorFlow log seviyesi azaltılır ve mevcut GPU aygıtları ekrana yazdırılır.
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
4. Hücre 1 - local source, Dizinler, Konfigürasyon ve Logging
4.1 local source package bağlantısı
Notebook Google Colab için hazırlanmıştır ve local source package, Deney1_Source_Runtime altına bağlanır.

force_remount=False kullanıldığı için local source zaten bağlıysa gereksiz yeniden bağlama yapılmaz.
from google.colab import local source
local source.mount("Deney1_Source_Runtime", force_remount=False)
4.2 Deney klasörünün güvenli bulunması
Kod önce beklenen standart yolu kontrol eder. Bu yol bulunamazsa hem LocalSource hem de shared local areas
altında Nazlıcan/Deney 1 dizini aranır. Aday klasörün geçerli sayılması için hem Kaş hem Sonuçlar alt klasörlerinin bulunması gerekir.
p.name == "Deney 1"
p.parent.name == "Nazlıcan"
(p / "Kaş").is_dir()
(p / "Sonuçlar").is_dir()
Hiç uygun klasör bulunamazsa FileNotFoundError; birden fazla uygun klasör bulunursa yanlış deney üzerinde çalışmayı önlemek için RuntimeError üretilir.
4.3 Sonuç klasörlerinin oluşturulması
Bulunan Deney 1 altında veri kaynağı Kaş, sonuç kökü ise Sonuçlar olarak tanımlanır. Yeni deney çıktıları

Sonuçlar/VGG16_FeatureExtractor_SVM_Kas altına yazılır.
Değişken Hedef
DATA_DIR Deney 1/Kaş
OUTPUT_DIR Deney 1/Sonuçlar/VGG16_FeatureExtractor_SVM_Kas
CHECKPOINTS_DIR Model ve scaler dosyaları
LOGS_DIR training.log
METRICS_DIR Metrik ve validation arama çıktıları
PREDICTIONS_DIR Frame/video tahminleri
FIGURES_DIR PNG grafikler
ARTIFACTS_DIR Feature cache ve dataset envanteri
Sayfa 5
4.4 CONFIG yapısı
Notebookun temel deney ayarları tek bir CONFIG sözlüğünde toplanır. Böylece model ayarları, etiketler, giriş boyutu ve SVM arama uzayı tek noktadan izlenebilir.
Ayar Değer / Anlam
labels real = 0, fake = 1
splits train, val, test
image_size 224 x 224
batch_size 32
VGG16 weights imagenet
include_top False
pooling avg
trainable False
feature_dim 512
SVM class_weight balanced
linear C 0.1, 1, 10
RBF C 1, 10, 100
RBF gamma scale, 0.001, 0.01
reuse_cached_features True
4.5 Veri dizini doğrulama ve logging
Model başlamadan önce real ve fake sınıflarının her biri için train, val ve test klasörleri kontrol edilir. Altı
klasörden biri eksikse eğitim durdurulur. Logging hem ekrana hem logs/training.log dosyasına yazılır. config_resolved.yaml dosyası da aynı aşamada oluşturulur.
5. Veri Envanteri ve Video Kimliği Çıkarımı

Kod .jpg, .jpeg, .png, .bmp ve .webp uzantılı görüntüleri recursive olarak tarar. Her frame için yol, sınıf,

sayısal etiket, split ve tahmini video_id bilgisi kayıt altına alınır.

```
records.append({
    "path": str(p),
    "class_name": class_name,
    "label": label,
    "split": split,
    "video_id": infer_video_id(p, split_dir),
})
```

infer_video_id fonksiyonu video kimliğini üç aşamalı bir kuralla tahmin eder:

1. Frame bir video alt klasörü içindeyse split altındaki ilk klasör adı video_id olur.
2. Alt klasör yoksa dosya adının sonundaki frame/frm + sayı deseni regex ile temizlenmeye çalışılır.
3. Bu da mümkün değilse dosyanın uzantısız adı doğrudan video_id olarak kullanılır.

Sonrasında pandas DataFrame ile train/val/test ve real/fake dağılımı özetlenir. Boş split veya tamamen eksik sınıf varsa deney durdurulur. Envanter artifacts/dataset_inventory.csv olarak kaydedilir.

6. Veri Sızıntısı / Split Kontrolleri

Notebook iki temel kontrol yapar. İlk kontrol, aynı dosya yolunun envanterde birden fazla kez yer alıp almadığıdır. Duplicate path bulunursa RuntimeError ile işlem durur.

İkinci kontrol, aynı sınıf + tahmini video_id kombinasyonunun kaç farklı splitte bulunduğunu sayar. Bir video kimliği birden fazla splitte görünürse deney tamamen durdurulmaz; kullanıcı uyarılır ve olası kayıtlar

artifacts/possible_video_leakage.csv dosyasına yazılır.

```
possible_video_leaks = video_split_counts[video_split_counts["split_count"] > 1]
```

Burada önemli nokta, video_id bilgisinin gerçek metadata üzerinden değil klasör/dosya adından tahmin edilmesidir. Dolayısıyla bu kontrol yardımcı bir güvenlik katmanıdır; video isimlendirme yapısı video kimliğini açık biçimde taşımıyorsa kesin bir doğrulama olarak yorumlanmamalıdır.

7. VGG16 Feature Extractor Yapısı

Bu deneyde VGG16 son sınıflandırıcı değildir. Modelin ImageNet üzerinde önceden öğrenilmiş convolution tabanı kullanılır; include_top=False ile orijinal sınıflandırma başlığı çıkarılır ve pooling="avg" ile son feature

haritaları global ortalama havuzlama yoluyla tek vektöre dönüştürülür.

```
feature_extractor = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3),
    pooling="avg",
)
feature_extractor.trainable = False
```

feature_extractor.trainable = False satırı nedeniyle VGG16 ağırlıkları bu deney sırasında güncellenmez. Yani kaş datasetine göre CNN fine-tuning yapılmaz; model yalnızca sabit bir derin özellik çıkarıcıdır. CONFIG ve feature_extractor.json içinde hedef feature boyutu 512 olarak tutulur.

8. Görüntü Ön İşleme, Batch İşleme ve Feature Cache

8.1 load_batch fonksiyonu

Her görüntü PIL ile açılır, RGB formata çevrilir, 224x224 boyutuna yeniden ölçeklenir ve float32 NumPy dizisine dönüştürülür. Ardından Keras VGG16 için preprocess_input uygulanır.

```
img = img.convert("RGB")
img = img.resize(IMG_SIZE)
arr = np.asarray(img, dtype=np.float32)
...
x = preprocess_input(x)
```

Okunamayan veya bozuk görüntüler tüm süreci durdurmak yerine errors listesine eklenir. Böylece batch pipeline hata toleranslı çalışır.

8.2 extract_features_for_split fonksiyonu

Fonksiyon train, val ve test için ayrı ayrı çağrılır. İlgili split envanterden seçilir, görüntüler batch_size=32 ile

işlenir ve her batch VGG16 predict çağrısından geçirilir.

```
feats = feature_extractor.predict(x, verbose=0)
```

Üretilen feature vektörleri, etiketler, görüntü yolları, video kimlikleri ve sınıf adları birlikte tutulur. Split

sonunda feature matrisleri birleştirilir ve sıkıştırılmış NPZ dosyasına yazılır.

8.3 Cache mekanizması

reuse_cached_features=True olduğu için daha önce train_features.npz, val_features.npz veya test_features.npz üretildiyse notebook önce cache kullanmayı dener. Cache yalnızca kayıtlı yolların ve etiketlerin güncel envanterle aynı olması durumunda yeniden kullanılır; aksi halde featurelar yeniden hesaplanır.

```
if cached_paths == current_paths and np.array_equal(cached_labels, current_labels):
    # cached features are reused
```

Bu mekanizma VGG16 feature extraction aşamasının her SVM denemesinde tekrar çalıştırılmasını

önleyerek sonraki çalıştırmaları hızlandırır.

9. SVM Hiperparametre Seçimi

9.1 Ölçekleme

Model seçimi aşamasında StandardScaler yalnızca train featureları üzerinde fit edilir. Validation setine yalnızca transform uygulanır. Böylece validation dağılımı scaler parametrelerini öğrenmek için kullanılmaz.

```
selection_scaler = StandardScaler()
X_train_scaled = selection_scaler.fit_transform(X_train)
X_val_scaled = selection_scaler.transform(X_val)
```

9.2 Aday modeller

Kod linear ve RBF kernel için sabit bir arama uzayı oluşturur. Linear SVM için üç C değeri, RBF SVM için üç C ve üç gamma değeri kombinasyonları denenir.

Kernel	C değerleri	Gamma değerleri
linear	0.1, 1, 10	scale
rbf	1, 10, 100	scale, 0.001, 0.01

Her aday model train üzerinde fit edilir; validation için accuracy, precision, recall, F1 ve ROC-AUC hesaplanır. fit_seconds ile eğitim süresi de kaydedilir.

9.3 En iyi modelin seçilme kuralı

Adaylar önce validation F1, eşitlik durumunda ROC-AUC, sonra accuracy değerine göre azalan biçimde sıralanır. Listenin ilk satırı best_params olarak seçilir. Tüm arama tablosu metrics/svm_validation_search.csv olarak dışa aktarılır.

```
search_df.sort_values(
    ["val_f1", "val_roc_auc", "val_accuracy"],
    ascending=False
)
```

10. Final Modelin Train + Validation ile Eğitilmesi

Hiperparametre seçimi tamamlandıktan sonra train ve validation featureları birleştirilir. Bu aşamada yeni bir

StandardScaler oluşturulur ve train+val birleşik setine fit edilir. Test setine yalnızca transform uygulanır.

```
X_trainval = np.concatenate([X_train, X_val], axis=0)
y_trainval = np.concatenate([y_train, y_val], axis=0)
```



```
final_scaler.fit_transform(X_trainval)
final_scaler.transform(X_test)
```

Final SVC, validation aşamasında seçilen kernel/C/gamma ile oluşturulur. class_weight="balanced" korunur ve final modelde predict_proba üretilebilmesi için probability=True yapılır.

Eğitimin sonunda final model checkpoints/svm_model.joblib, scaler ise checkpoints/scaler.joblib olarak kaydedilir. Eğitim süresi final_fit_seconds değişkeninde tutulur.

11. Test Tahminleri ve Frame-Level Metrikler

Test featureları final scaler ile dönüştürüldükten sonra SVM ile sınıf tahmini ve sınıf olasılıkları alınır. Kod

final_svm.classes_ içinden FAKE=1 sınıfının hangi probability sütununda bulunduğunu dinamik olarak bulur.

```
test_pred = final_svm.predict(X_test_scaled)
test_proba_all = final_svm.predict_proba(X_test_scaled)
...
test_fake_prob = test_proba_all[:, fake_col]
```

Ayrıca decision_function alınmaya çalışılır; herhangi bir hata olması halinde decision_score olarak fake_probability kullanılır. Böylece predictions CSV dosyasında karar skorunun boş kalması önlenir.

Metrik / çıktı	Kodun ürettiği anlam
Accuracy	Tüm test frameleri içinde doğru sınıflandırma oranı.
Precision	FAKE tahminlerinin ne kadarının gerçekten FAKE olduğu.
Recall	Gerçek FAKE framelerin ne kadarının yakalandığı.
F1-score	Precision ve recall dengesini tek değerde özetleyen skor.
ROC-AUC	FAKE olasılık skorları üzerinden ayrıştırma performansını özetler.
Average Precision	Precision-recall eğrisini tek bir özet skora indirger.
Classification report	REAL ve FAKE için sınıf bazlı precision, recall, F1 ve support.
Confusion matrix	true/pred REAL-FAKE sayımlarını 2x2 tablo olarak gösterir.

Her test frame için görüntü yolu, video_id, gerçek sınıf, tahmin edilen sınıf, fake_probability ve decision_score predictions/test_predictions.csv dosyasına yazılır.

12. Video-Level Değerlendirme Mantığı

Notebook, test tahminlerini true_label + true_class + video_id bazında gruplar ve aynı video altında birden

fazla frame varsa fake_probability değerlerinin ortalamasını alır.

```
groupby(["true_label", "true_class", "video_id"])
.agg(fake_probability=("fake_probability", "mean"),
    frame_count=("image_path", "size"))
```

Video-level değerlendirme ancak iki koşul birlikte sağlanırsa gerçekleştirilir: gruplama sonucunda frame sayısından daha az video kaydı oluşmalı ve iki gerçek sınıf da bulunmalıdır. Ortalama FAKE olasılığı 0.5 veya üstündeyse video FAKE kabul edilir. Koşullar sağlanmazsa video_metrics=None kalır ve notebook video-level aggregation atlandı mesajı verir.

13. Grafiklerin Üretilmesi

Notebook test değerlendirmesinden sonra üç PNG görsel oluşturur ve figures klasörüne kaydeder:

☐ confusion_matrix.png: 2x2 gerçek/tahmin sınıf sayılarını görselleştirir.
☐ roc_curve.png: false positive rate - true positive rate ilişkisini ve ROC-AUC değerini gösterir.
☐ precision_recall_curve.png: recall - precision ilişkisini ve Average Precision değerini gösterir.

Görseller 180 DPI ile ve bbox_inches="tight" kullanılarak kaydedilir. Bu grafikler model raporunda doğrudan kullanılabilecek şekilde ayrı dosyalar olarak üretilir.

14. Deney Metadata ve Tekrarlanabilirlik Dosyaları

14.1 metrics.json

Deney adı, bölge, feature extractor tanımı, classifier, seçilen SVM parametreleri, frame-level ve varsa video-level metrikleri tek JSON dosyasında toplanır.

14.2 environment.json

UTC zaman damgası, Python sürümü, platform, TensorFlow, NumPy, pandas, scikit-learn sürümleri, GPU listesi ve seed değeri kaydedilir.

14.3 requirements_lock.txt

pip freeze çıktısı kaydedilerek deney çalıştığı anda ortamda yüklü paketlerin sürümleri arşivlenir. Komut

başarısız olursa hata metni dosyaya yazılır.

14.4 run_summary.json

Deney tamamlandı bilgisi, dataset/output yolları, split-sınıf adetleri, feature matris boyutları, en iyi SVM
parametreleri, final eğitim süresi ve metrikler bir özet dosyasında tutulur.

15. Output Manifest ve Dosya Bütünlüğü

Notebook deney klasöründeki tüm dosyaları recursive biçimde tarar. Her dosya için göreli yol, bayt cinsinden boyut ve SHA-256 özeti hesaplanır. output_manifest.csv dosyasının kendisi hash listesine dahil edilmez.

relative_path | size_bytes | sha256

Bu manifest, deney çıktılarının hangi dosyalardan oluştuğunu kayıt altına alır ve daha sonra dosyaların değişip değişmediğini kontrol etmeyi kolaylaştırır.

16. Üretilen Dosyaların Anlamı

Dosya / klasör	İçerik ve kullanım amacı
checkpoints/svm_model.joblib	Final eğitilmiş SVM sınıflandırıcısı.
checkpoints/scaler.joblib	Final train+val üzerinde fit edilmiş StandardScaler.
logs/training.log	Deney akışı, feature boyutları, seçilen parametreler ve bilgilendirme mesajları.
metrics/metrics.json	Temel deney ve performans özeti.
metrics/classification_report.csv	REAL/FAKE sınıf bazlı değerlendirme.
metrics/confusion_matrix.csv	Sayısal confusion matrix.

metrics/svm_validation_search.csv	Tüm SVM adaylarının validation sonuçları.
predictions/test_predictions.csv	Her test frame için tahmin, skor ve metadata.
predictions/video_level_predictions.csv	Video grupları oluşursa video bazlı tahminler.
figures/*.png	Confusion matrix, ROC, precision-recall grafikleri.
artifacts/*_features.npz	VGG16 ile çıkarılan train/val/test feature cacheleri.
artifacts/dataset_inventory.csv	Kullanılan her görüntünün sınıf/split/video_id envanteri.
artifacts/feature_extractor.json	VGG16 feature extractor teknik tanımı.
config_resolved.yaml	Deney yapılandırmasının çözülmüş hali.
environment.json	Çalışma ortamı ve sürüm bilgileri.
requirements_lock.txt	Kurulu Python paketlerinin pip freeze çıktısı.
run_summary.json	Deney akışının kısa makine-okunabilir özeti.
output_manifest.csv	Çıktı dosyaları, boyutları ve SHA-256 özetleri.

17. Kodun Güçlü Tasarım Noktaları

☐ Mevcut train/val/test ayrımını korur; notebook içinde yeniden rastgele split üretmez.
☐ Validation, hiperparametre seçimi için ayrılmıştır; final test değerlendirmesi ayrı tutulur.
☐ Final scaler test verisi üzerinde fit edilmez.
☐ Feature extraction ile SVM sınıflandırması birbirinden ayrılmıştır; VGG16 özellikleri cachelenebilir.
☐ Bozuk görüntüler tek başına tüm feature extraction sürecini durdurmaz; hata listesi üretilebilir.
☐ Çıktılar sistematik klasörlere ayrılır ve model/scaler kaydedilir.
☐ Deney yapılandırması, paket sürümleri, ortam bilgisi ve manifest arşivlenir.
☐ Olası video bazlı split çakışması için ek kontrol katmanı bulunur.

18. Koddan Doğrudan Görülen Sınırlamalar ve Dikkat Noktaları
Aşağıdaki maddeler notebookun mevcut kod yapısından doğrudan çıkarılabilen teknik dikkat noktalarıdır; bunlar deney sonucunun başarım yorumundan bağımsızdır.
☐ VGG16 tamamen dondurulmuştur. Kod, kaş verisi üzerinde CNN fine-tuning yapmaz.

☐ Görüntüler doğrudan 224x224 boyutuna resize edilir. Kod içinde aspect-ratio koruyan padding/crop mekanizması bulunmaz.
☐ Video kimliği metadata dosyasından okunmaz; klasör adı veya dosya adı üzerinden tahmin edilir. Bu nedenle video-level değerlendirme veri adlandırmasına bağlıdır.
☐ Cache geçerlilik kontrolü görüntü yolu ve etikete dayanır. Aynı path altında görüntü içeriği değişirse cache bunu içerik hash ile tespit etmez.
☐ Olası video leakage bulunduğunda deney otomatik durdurulmaz; yalnızca uyarı ve CSV üretir.
☐ SVM arama uzayı kod içinde önceden belirlenmiş sınırlı kombinasyonlardan oluşur; otomatik geniş optimizasyon yapılmaz.
☐ Video-level karar eşiği kodda sabit 0.5 olarak kullanılır.
☐ Final probability üretimi SVC(probability=True) üzerinden yapılır; notebook ayrıca ayrı bir olasılık kalibrasyon adımı tanımlamaz.
☐ training.log FileHandler mode="w" ile açıldığı için aynı output klasöründe yeni çalıştırma önceki log dosyasını üzerine yazabilir.

19. Özet Pseudocode
1. Gerekli paketleri kur ve import et.
2. Seed = 42 ayarla; GPU bilgisini göster.
3. local source package'i bağla.
4. Nazlıcan/Deney 1 klasörünü bul.
5. Kaş veri dizinini ve Sonuçlar/VGG16_FeatureExtractor_SVM_Kas çıktı dizinini tanımla.
6. real/fake -> train/val/test klasörlerinin tamamını doğrula.
7. Dataset envanterini oluştur; her frame için path, class, label, split, video_id kaydet.
8. Duplicate path ve olası video split leakage kontrollerini yap.
9. ImageNet VGG16'yi include_top=False, pooling=avg, trainable=False ile yükle.
10. Train, val ve test framelerini 224x224 RGB biçiminde batchlerle oku.
11. preprocess_input uygula ve VGG16 ile 512-D feature çıkar.
12. Featureları split bazında NPZ cache olarak kaydet.
13. StandardScaler'i train üzerinde fit et.
14. Linear/RBF SVM adaylarını train üzerinde eğit; validation metriklerini hesapla.
15. F1 -> ROC-AUC -> Accuracy önceliğine göre en iyi parametreyi seç.
16. Train + val featurelarını birleştir.
17. Yeni scaler'i train+val üzerinde fit et; test'e yalnızca transform uygula.
18. Final SVM'i seçilen parametrelerle train+val üzerinde eğit.
19. SVM ve scaler'i checkpoints klasörüne kaydet.
20. Test tahminleri, olasılıklar, metrikler, classification report ve confusion matrix üret.
21. Mümkünse frame skorlarını video_id bazında ortalayıp video-level metrik hesapla.
22. Confusion matrix, ROC ve precision-recall grafiklerini kaydet.
23. metrics.json, environment.json, requirements_lock.txt ve run_summary.json üret.
24. Tüm çıktı dosyaları için SHA-256 içeren output_manifest.csv oluştur.
Sonuç
Bu notebookun temel yaklaşımı, kaş bölgesindeki görüntü bilgisini VGG16 üzerinden sabit bir 512 boyutlu derin temsil haline getirmek ve REAL/FAKE ayrımını SVM ile gerçekleştirmektir. Kod yalnızca model eğitimini değil; dataset doğrulama, feature caching, model seçimi, final test, görselleştirme, model dosyalarının saklanması ve deney tekrarlanabilirliği için gerekli metadata üretimini de tek bir pipeline içinde
toplar. Bu nedenle notebook, tek seferlik bir eğitim betiğinden çok, izlenebilir ve yeniden çalıştırılabilir bir
deney pipelinei olarak tasarlanmıştır.
Kaynak notu: Bu rapor, yüklenen VGG16_FeatureExtractor_SVM_Kas_Deney1_FIXED (1).ipynb notebookunun kod akışı temel alınarak hazırlanmıştır.